

Peramalan kadar NO_2 di daerah Bangkalan Madura (revisi)

Contents

- Latar Belakang
- 1. Pengumpulan Data
- 2. Preprocessing Data
- 3. Modeling menggunakan KNN Regression

Latar Belakang

Peningkatan aktivitas industri, transportasi, serta pertumbuhan populasi yang pesat telah menyebabkan peningkatan signifikan terhadap tingkat pencemaran udara di berbagai wilayah. Salah satu polutan udara utama yang menjadi perhatian adalah Nitrogen Dioksida (NO_2), yaitu gas beracun yang dihasilkan terutama dari proses pembakaran bahan bakar fosil seperti kendaraan bermotor, pembangkit listrik, dan kegiatan industri. NO_2 memiliki dampak serius terhadap kesehatan manusia, seperti gangguan pernapasan, iritasi paru-paru, serta memperburuk penyakit asma dan bronkitis. Selain itu, NO_2 juga berkontribusi terhadap pembentukan hujan asam dan penurunan kualitas lingkungan secara keseluruhan.

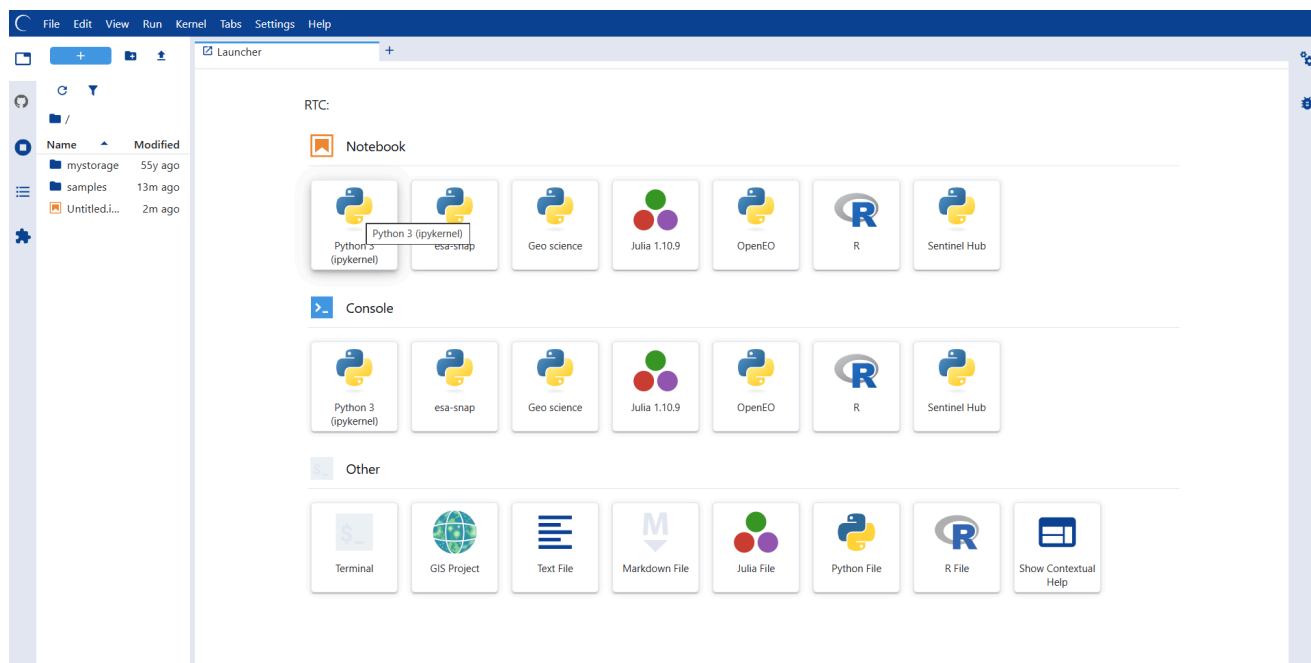
1. Pengumpulan Data

Pertama kita akan mengumpulkan data Time Series Harian kadar NO_2 di daerah Bangkalan. Pengumpulan data dari sumber website <https://dataspace.copernicus.eu/>, buat akun terlebih dahulu di website copernicus tersebut.

Dokumentasi cara pengambilan data di

<https://documentation.dataspace.copernicus.eu/notebook-samples/openeo/NO2Covid.html>

Untuk menuliskan code Python untuk mengambil data, silahkan kunjungi halaman <https://dataspace.copernicus.eu/analyse/jupyterlab>, klik Access JupyterLab, scroll kebawah sedikit ..., lalu pilih Python 3 (ipykernel)



Disini kita akan mengambil data kadar NO_2 di daerah Bangkalan dari tanggal ... sampai

Kita install terlebih dahulu openoneo:

```
pip install openeo
```

Lalu tuliskan code dibawah:

```
import openeo
```

```
connection = openeo.connect("openeo.dataspace.copernicus.eu").authenticate_oidc()
```

pada saat menjalankan baris code diatas (connection), nanti akan diminta autentikasi seperti output berikut:

```
Terminal/Output
Visit (link autentikasi) to authenticate.
✅ Authorized successfully
Authenticated using device code flow.
```

Kalian tinggal klik link autentikasi lalu login menggunakan akun "copernicus" kalian.

```

aoi = {
    "type": "Polygon",
    "coordinates": [
        [
            [113.09, -6.89],
            [112.68, -6.89],
            [112.68, -7.20],
            [113.09, -7.20],
            [113.09, -6.89],
        ]
    ]
}

s5post = connection.load_collection(
    "SENTINEL_5P_L2",
    temporal_extent=["2023-10-01", "2025-10-01"],
    spatial_extent={
        "west": 112.68,
        "south": -7.20,
        "east": 113.09,
        "north": -6.89
    },
    bands=["NO2"],
)

# Now aggregate by day to avoid having multiple data per day
s5p_no2_daily = s5post.aggregate_temporal_period(reducer="mean", period="day")

# Now create a spatial aggregation to generate mean timeseries data
s5p_no2_aoi = s5p_no2_daily.aggregate_spatial(reducer="mean", geometries=aoi)

```

Code diatas memerlukan titik koordinasi area yang akan diambil data NO_2 -nya, untuk mengambil titik koordinasi kaian kunjungi webiste

<https://geojson.io/#map=14.8/-7.04732/112.69463> . Didalam website tersebut kalian akan memilih daerah dengan cara memberi shape kotak didaerah yang ingin kalian ambil datanya.

The screenshot shows the geojson.io interface. On the left is a map of the Bangkalan area in Madura, Indonesia, with a rectangular selection box. On the right is a JSON editor showing the data for the selected area. The JSON is a FeatureCollection containing a single Feature with a polygon geometry. The coordinates of the polygon are listed in the JSON.

```

1  {
2    "type": "FeatureCollection",
3    "features": [
4      {
5        "type": "Feature",
6        "properties": {},
7        "geometry": {
8          "coordinates": [
9            [
10             [
11              113.07680309239902,
12              -6.877875870487117
13            ],
14            [
15              112.67449667661083,
16              -6.877875870487117
17            ],
18            [
19              112.67449667661083,
20              -7.187461929031443
21            ],
22            [
23              113.07680309239902,
24              -7.187461929031443
25            ],
26            [
27              113.07680309239902,
28              -6.877875870487117
29            ]
30          ]
31        }
32      }
33    ]
34  }

```

Di panel sebelah kanan terdapat data JSON yang berupa koordinat daerah yang kalian pilih, kalian salin terus sesuaikan dengan code diatas di bagian variabel "aoi" dan spatial_extent.

Lalu kalian tambahkan baris code dibawah untuk memulai pengambilan data:

```
job = s5post.execute_batch(title="N02 in Bangkalan", outputfile="N02Bangkalan.nc")
```

Tunggu proses pengambilan data, output proses seperti berikut:

```

0:00:00 Job 'j-2510231608434524a87dedeacfaf5a43': send 'start'
0:00:15 Job 'j-2510231608434524a87dedeacfaf5a43': created (progress 0%)
0:00:20 Job 'j-2510231608434524a87dedeacfaf5a43': created (progress 0%)
0:00:26 Job 'j-2510231608434524a87dedeacfaf5a43': created (progress 0%)
0:00:35 Job 'j-2510231608434524a87dedeacfaf5a43': queued (progress 0%)
0:00:46 Job 'j-2510231608434524a87dedeacfaf5a43': queued (progress 0%)
0:00:58 Job 'j-2510231608434524a87dedeacfaf5a43': queued (progress 0%)
0:01:14 Job 'j-2510231608434524a87dedeacfaf5a43': queued (progress 0%)
0:01:33 Job 'j-2510231608434524a87dedeacfaf5a43': running (progress N/A)
0:01:57 Job 'j-2510231608434524a87dedeacfaf5a43': running (progress N/A)
0:02:27 Job 'j-2510231608434524a87dedeacfaf5a43': running (progress N/A)
0:03:05 Job 'j-2510231608434524a87dedeacfaf5a43': running (progress N/A)
0:03:52 Job 'j-2510231608434524a87dedeacfaf5a43': running (progress N/A)
0:04:50 Job 'j-2510231608434524a87dedeacfaf5a43': running (progress N/A)
0:05:50 Job 'j-2510231608434524a87dedeacfaf5a43': running (progress N/A)
0:06:50 Job 'j-2510231608434524a87dedeacfaf5a43': running (progress N/A)
0:07:50 Job 'j-2510231608434524a87dedeacfaf5a43': finished (progress 100%)

```

Abaikan ketika ada N/A.

Ketika proses pengambilan data, aktivitas kalian akan terekam di halaman

<https://editor.openeo.org/?>

[server=https%3A%2F%2Fopeneo.dataspace.copernicus.eu%2Fopeneo%2F1.2](https%3A%2F%2Fopeneo.dataspace.copernicus.eu%2Fopeneo%2F1.2) . Disitu terdapat nama dataset dan status pengambilan data.

The screenshot shows the OpenEO Web Editor interface. The top bar includes the OpenEO logo, version 0.14.0, and navigation links for Help, Wizard, Server, and the user profile (Maulana Ardiansyah). On the left, there is a sidebar with a search bar and filters for 'Show' (depreciated, experimental). Below this, a list of collections and processes is shown: Collections (20), Processes (117/138), UDF Runtimes (2), and Export File Formats (9).

The main area displays a 'Welcome!' message and instructions on how to build a model using the visual model builder. It also provides instructions on how to import existing processes into the model builder, including pasting JSON from the clipboard, dragging and dropping a JSON file, or importing a JSON file from the internet.

Below the welcome message, there is a section for 'Data Processing' and 'User-Defined Processes'. A table lists the batch jobs, showing their status, submitted time, and last update time. The table has columns for Batch Job, Status, Submitted, Last update, and Actions.

Batch Job	Status	Submitted	Last update	Actions
NO2 in Bangkalan	finished	10/23/2025, 4:08:43 PM UTC	10/23/2025, 4:16:10 PM UTC	[Icons]
NO2 in Kwanyar	finished	10/23/2025, 3:50:18 PM UTC	10/23/2025, 3:56:13 PM UTC	[Icons]
NO2 in Bangkalan	error	10/23/2025, 3:32:47 PM UTC	10/23/2025, 3:42:13 PM UTC	[Icons]
NO2 Bangkalan	error	10/23/2025, 3:11:39 PM UTC	10/23/2025, 3:19:12 PM UTC	[Icons]
NO2 Bangkalan	error	10/23/2025, 3:03:58 PM UTC	10/23/2025, 3:06:13 PM UTC	[Icons]

2. Preprocessing Data

Setelah kita mengambil data, data bisa diunduh di halaman <https://editor.openeo.org/?server=https%3A%2F%2Fopeneo.dataspace.copernicus.eu%2Fopeneo%2F1.2> . File akan berbentuk .nc. Kita cuman perlu kolom date dan NO2 menggunakan code dibawah:

```

import netCDF4

file_path = "data/NO2Bangkalan.nc"
ds = netCDF4.Dataset(file_path)

# Lihat seluruh variabel yang tersedia
print("📦 Variabel dalam file:")
print(ds.variables.keys())
# dict_keys(['t', 'x', 'y', 'crs', 'NO2'])

# Ambil NO2
no2 = ds.variables["NO2"][:]

# Ambil Time
time = ds.variables["t"][:]

# Konversi waktu ke format tanggal jika punya atribut 'units'
try:
    time_units = ds.variables["t"].units
    dates = netCDF4.num2date(time, units=time_units)
except Exception:
    dates = time # fallback kalau tidak ada units

# Tampilkan struktur data NO2
print(type(no2))
# type <class 'numpy.ma.core.MaskedArray'>

print(len(no2))
# banyaknya data record NO2 725

print(len(no2[0]))
# panjang data perbaris 9

print(len(no2[0][0]))
# panjang perdata 8

print(no2[0][0][0])
# 3.7701793e-05

```

Dari code diatas kita mengetahui bentuk data dari kolom NO2 nya.

jadi struktur data NO2 perbaris adalah:

```

[
    [[ * 8] * 9
]

```

Untuk melihat 10 data pertama adalah:

```

print("Contoh data pertama:")
for i in range(0, 10):
    print(no2[i])

```

Dalam sehari, terdapat banyak data NO_2 , jadi kita rata-ratakan agar satu cell data hanya terdapat satu value. Namun terdapat masalah pada data NO_2 seperti missing value. Contoh pada output dibawah:

Terminal/Output

```
[2.9651806471520104e-05 4.1052295273402706e-05 -- 5.6563803809694946e-05  
-- -- 6.348737952066585e-05 --]
```

a. Mengatasi Missing Value menggunakan metode Interpolasi Linear

Sekarang kita akan mengatasi permasalahan missing value pada data NO_2 .

```
import numpy as np
import pandas as pd

# Interpolasi Linear
no2_filled = np.zeros_like(no2)
# Untuk jaga-jaga jika terdapat '--' tidak berubah menjadi 0
no2_filled = no2_filled.filled(0)

# loop tiap grid (y,x)
for i in range(no2.shape[1]): # 9 baris
    for j in range(no2.shape[2]): # 8 kolom
        series = pd.Series(no2[:, i, j])
        no2_filled[:, i, j] = series.interpolate(method='linear', limit_direction='
```

Dengan code diatas, missing value yang terdapat pada data NO_2 akan diisi secara otomatis menggunakan metode Interpolasi Linear.

b. Rata-rata kan Data dan ubah Datetime

Setelah mengatasi missing value, kita akan me-rata-rata-kan data NO_2 agar satu record hanya berupa single value. Sekalian kita mengambil date nya dan menaruh di array. Kita akan mengubah datetime dari awalnya (2023-10-04 00:00:00) menjadi (2023-10-04) karena kita mengambil data time series harian jadi kita tidak memerlukan data jam, menit dan detik.

```
new_dates = []
new_no2 = []
for i in range(len(dates)):
    # ubah format datetime
    new_date = dates[i].strftime('%Y-%m-%d')
    new_dates.append(new_date)
    new_no2.append(np.mean(no2_filled[i]))
```

c. Simpan data dalam bentuk CSV

Setelah itu kita akan membentuk data menjadi DataFrame Pandas untuk disimpan menjadi CSV.

```
df = pd.DataFrame({
    "date": dates,
    "NO2": no2_values
})

# Simpan ke CSV
df.to_csv("NO2_Bangkalan_timeseries.csv", index=False)
```

Untuk mengatasi missing value dan menyimpan data ke CSV sudah berhasil.

d. Pengecekan Missing Value data harian pada CSV

Sekarang setelah data berbentuk CSV, kita cek apakah data Time Series harian lengkap. Cara men-cek apakah data Time Series Harian lengkap gunakan code dibawah:


```
import pandas as pd
import numpy as np

df = pd.read_csv("NO2_Bangkalan_timeseries.csv")

# Pastikan kolom 'date' bertipe datetime
df['date'] = pd.to_datetime(df['date'])

# Buat rentang tanggal lengkap
start_date = "2023-10-01"
end_date = "2025-09-30"
full_range = pd.date_range(start=start_date, end=end_date, freq='D')

# Cek tanggal yang hilang
missing_dates = full_range.difference(df['date'])

print(f"Jumlah hari missing: {len(missing_dates)}")
print("Daftar tanggal missing:")
print(missing_dates)
```

```
output/terminal
Jumlah hari missing: 6
Daftar tanggal missing:
DatetimeIndex(['2023-11-11', '2024-01-01', '2024-03-23', '2024-08-12',
               '2025-01-30', '2025-01-31'],
              dtype='datetime64[ns]', freq=None)
```

Dalam kasus saya ini, terdapat 6 hari missing value. Kita akan mengatasi lagi missing value menggunakan metode Interpolasi Linear. Cara memperbaikinya gunakan code dibawah:

```
import pandas as pd

# Pastikan datetime dan sorting
df['date'] = pd.to_datetime(df['date'])
df = df.sort_values('date')

# Buat rentang tanggal lengkap
full_range = pd.date_range(start="2023-10-01", end="2025-09-30", freq='D')

# Reindex agar tanggal yang hilang muncul sebagai NaN
df = df.set_index('date').reindex(full_range)
df.index.name = 'date'

# Interpolasi linear berdasarkan indeks waktu
df['NO2'] = df['NO2'].interpolate(method='time')

# (Opsional) jika masih ada NaN di bagian awal/akhir bisa gunakan forward/backward
df['NO2'] = df['NO2'].fillna(method='bfill').fillna(method='ffill')

# Simpan kembali ke CSV
df.to_csv("no2_timeseries_interpolated.csv")
```

Setelah saya cek missing value harian, sudah tidak ada lagi missing value.

```
Jumlah hari missing: 0
Daftar tanggal missing:
DatetimeIndex([], dtype='datetime64[ns]', freq='D')
```

dengan bentuk data terdapat 2 kolom, kolom pertama yaitu date atau tanggal, kolom kedua yaitu kadar NO_2 yang sudah di rata-rata kan.

```
      date      NO2
0  2023-10-01  0.000027
1  2023-10-02  0.000024
2  2023-10-03  0.000024
3  2023-10-04  0.000021
4  2023-10-05  0.000021
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 731 entries, 0 to 730
Data columns (total 2 columns):
 #   Column  Non-Null Count  Dtype  
---  -
 0   date    731 non-null     object  
 1   NO2     731 non-null     float64
dtypes: float64(1), object(1)
memory usage: 11.5+ KB
```

e. Deteksi Outlier IQR

Setelah kita mengisi missing value menggunakan metode Interpolasi Linear, selanjutnya kita akan mendeteksi Outlier menggunakan metode IQR.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

df = pd.read_csv("no2_timeseries_interpolated.csv")

df['date'] = pd.to_datetime(df['date'])

# Hitung IQR
Q1 = df['NO2'].quantile(0.25)
Q3 = df['NO2'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

# Filter outlier
outliers_iqr = df[(df['NO2'] < lower_bound) | (df['NO2'] > upper_bound)]

print("Jumlah Outlier (IQR):", len(outliers_iqr))
print(outliers_iqr[['date', 'NO2']].head())
```

output/terminal

```
Jumlah Outlier (IQR): 14
   date      NO2
45 2023-11-15  0.000051
46 2023-11-16  0.000044
48 2023-11-18  0.000051
67 2023-12-07  0.000047
68 2023-12-08  0.000045
```

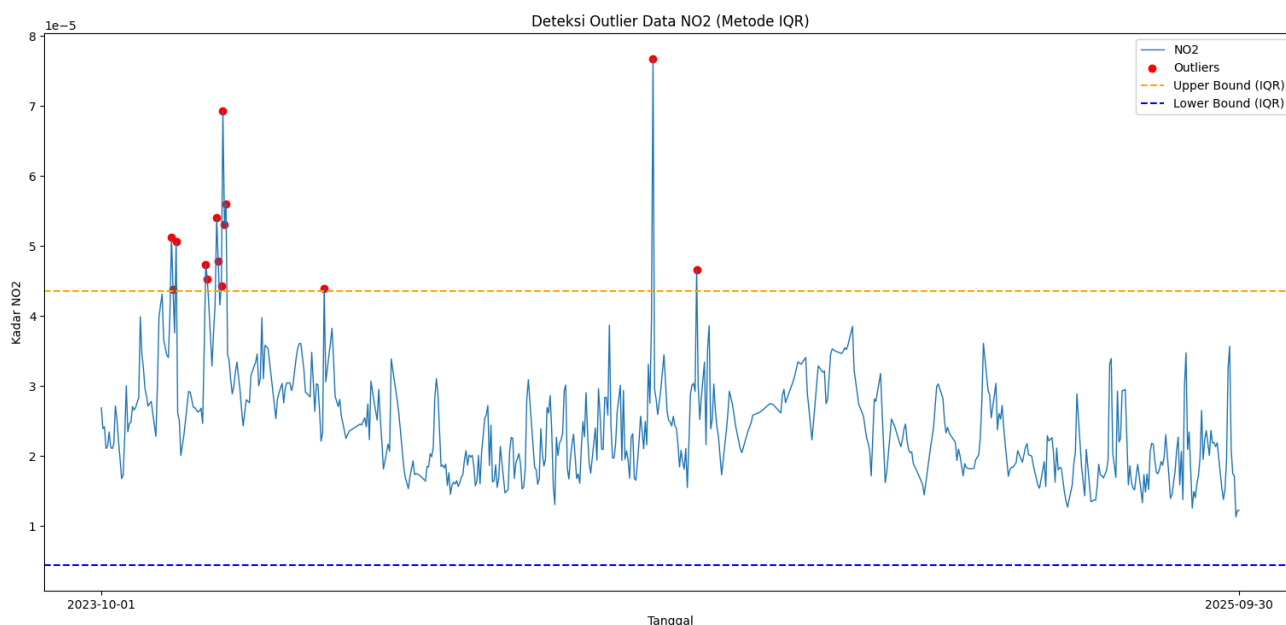
Untuk men-visualisasi outlier:

```
# === Visualisasi ===
plt.figure(figsize=(15,5))
plt.plot(df['date'], df['NO2'], label="NO2", linewidth=1)

# Titik Outlier
plt.scatter(outliers_iqr['date'], outliers_iqr['NO2'],
            color='red', marker='o', label="Outliers")

# Garis batas atas & bawah
plt.axhline(upper_bound, color='orange', linestyle='dashed', label="Upper Bound (IQR)")
plt.axhline(lower_bound, color='blue', linestyle='dashed', label="Lower Bound (IQR)")

plt.title("Deteksi Outlier Data NO2 (Metode IQR)")
plt.xlabel("Tanggal")
plt.ylabel("Kadar NO2")
plt.legend()
plt.tight_layout()
plt.xticks(
    ticks=[df['date'].iloc[0], df['date'].iloc[-1]],
    labels=[df['date'].iloc[0].strftime('%Y-%m-%d'),
            df['date'].iloc[-1].strftime('%Y-%m-%d')]
)
plt.show()
```



Setelah itu, kita akan menghapus data outlier. Karena data ini merupakan data Time Series, maka data outlier yang dihapus akan diisi kembali menggunakan Interpolasi Linear.

```
# Tandai outlier menjadi NaN
df['NO2_cleaned'] = df['NO2'].mask((df['NO2'] < lower_bound) | (df['NO2'] > upper_bound))

print("Jumlah nilai yang dinyatakan sebagai outlier:", df['NO2_cleaned'].isna().sum())

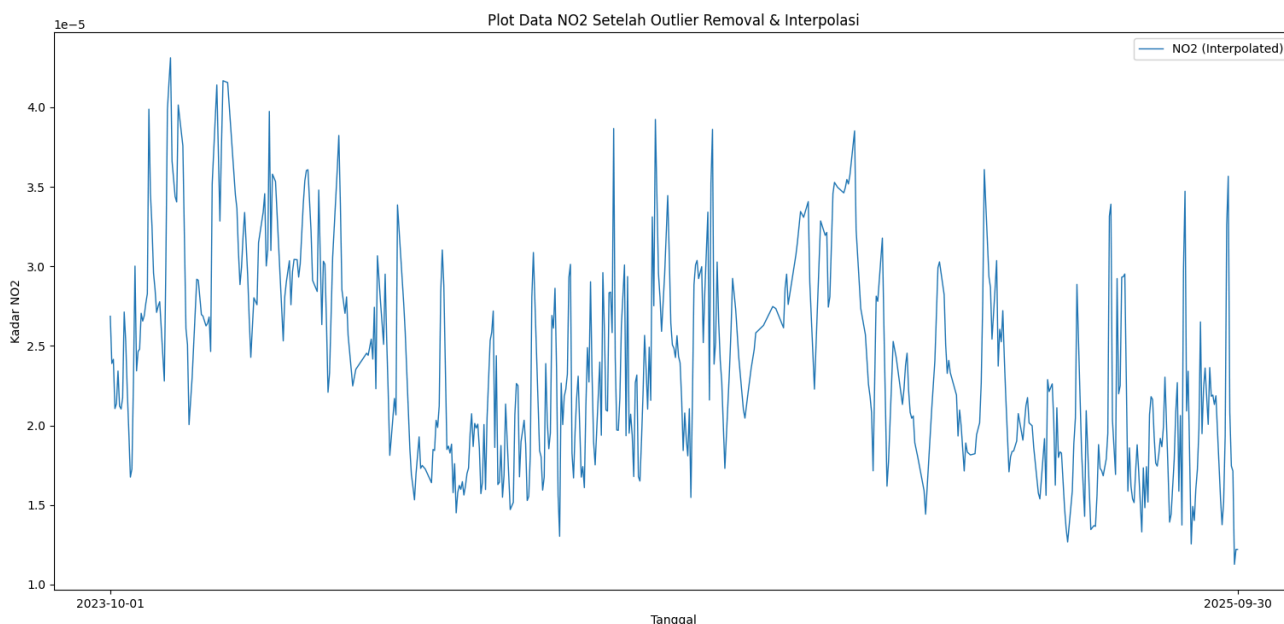
# Interpolasi linear untuk mengisi kembali nilai outlier
df['NO2_filled'] = df['NO2_cleaned'].interpolate(method='linear')

# Jika masih tersisa NaN di ujung data, isi dengan forward/backward fill
df['NO2_filled'] = df['NO2_filled'].bfill().ffill()
# df['NO2_filled'] = df['NO2_filled'].fillna(method='bfill').fillna(method='ffill')

print("Jumlah missing setelah interpolasi:", df['NO2_filled'].isna().sum())
```

Visualisasi data setelah menghapus Outlier dan mengisi kembali menggunakan Interpolasi Linear:

```
plt.figure(figsize=(15,5))
# Plot data hasil interpolasi
plt.plot(df['date'], df['NO2_filled'], label="NO2 (Interpolated)", linewidth=1)
# Tampilkan hanya tanggal awal dan akhir di sumbu X
plt.xticks(
    ticks=[df['date'].iloc[0], df['date'].iloc[-1]],
    labels=[df['date'].iloc[0].strftime('%Y-%m-%d'),
            df['date'].iloc[-1].strftime('%Y-%m-%d')]
)
plt.title("Plot Data NO2 Setelah Outlier Removal & Interpolasi")
plt.xlabel("Tanggal")
plt.ylabel("Kadar NO2")
plt.legend()
plt.tight_layout()
plt.show()
```



3. Modeling menggunakan KNN Regression

Dengan data Time Series kadar NO_2 harian di daerah Bangkalan, kita akan memprediksi kadar NO_2 satu hari yang akan datang. Sekarang kita akan ubah data, mencoba mencari korelasi antara 1 hari dengan 4 hari sebelumnya. Kita juga akan membandingkan apakah semakin banyak hari sebelumnya, model akan lebih bagus?

a. Uji Korelasi Data

Sebelum masuk ke modeling, data kita merupakan data unsupervised yang berarti tidak ada label. Kita ubah data menjadi supervised lalu uji korelasi terhadap label (t). Fitur-fitur nya merupakan 30 hari sebelum ($t-30$, $t-29$, ... $t-1$) dan label (t).

```
import pandas as pd

def create_supervised(data, n_lag=4):
    df_supervised = pd.DataFrame()

    # Membuat fitur t-4 sampai t-1
    for i in range(n_lag, 0, -1):
        df_supervised[f'N02(t-{i})'] = data.shift(i)

    # Label hari H
    df_supervised['N02(t)'] = data

    # Hapus baris yang masih mengandung NaN akibat shift
    df_supervised.dropna(inplace=True)

    return df_supervised

# contoh penggunaan
supervised_df30 = create_supervised(df['N02_scaled'], n_lag=30)

# Ambil semua lag dan kolom target
lag_cols = supervised_df30.drop(columns="N02(t)").columns
correlations = supervised_df30[lag_cols].corrwith(supervised_df30['N02(t)'])

# Tampilkan nilai korelasi
print(correlations)
```

output/terminal

```
N02(t-30)    0.442365
N02(t-29)    0.454480
N02(t-28)    0.475354
N02(t-27)    0.411464
N02(t-26)    0.381559
N02(t-25)    0.368824
N02(t-24)    0.353114
N02(t-23)    0.364938
N02(t-22)    0.372437
N02(t-21)    0.380476
N02(t-20)    0.350856
N02(t-19)    0.342492
N02(t-18)    0.312603
N02(t-17)    0.283336
N02(t-16)    0.288346
N02(t-15)    0.292171
N02(t-14)    0.311974
N02(t-13)    0.327142
N02(t-12)    0.341764
N02(t-11)    0.374090
N02(t-10)    0.397377
N02(t-9)     0.419258
N02(t-8)     0.455909
N02(t-7)     0.462456
N02(t-6)     0.460161
N02(t-5)     0.491515
N02(t-4)     0.523820
N02(t-3)     0.593839
```

```
NO2(t-2)    0.675955
NO2(t-1)    0.796441
```

Skala nilai uji korelasi itu dari -1 sampai 1, namun kita ambil nilai uji korelasi yang terbaik yaitu lebih dari 0.5 yaitu fitur t-1 sampai t-4.

b. Normalisasi Data

karena kita menggunakan model KNN Regression, maka perlu normalisasi data menggunakan min-max Scaler.

```
from sklearn.preprocessing import MinMaxScaler
import pandas as pd

scaler = MinMaxScaler()

df['NO2_scaled'] = scaler.fit_transform(df[['NO2']])
```

Maka data akan di-normalisasi 0-1.

```
output/terminal
      date      NO2  NO2_scaled
0  2023-10-01  0.000027   0.238203
1  2023-10-02  0.000024   0.192840
2  2023-10-03  0.000024   0.196854
3  2023-10-04  0.000021   0.149560
4  2023-10-05  0.000021   0.154247
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 731 entries, 0 to 730
Data columns (total 3 columns):
 #   Column      Non-Null Count  Dtype  
---  -
 0   date        731 non-null    object  
 1   NO2         731 non-null    float64 
 2   NO2_scaled  731 non-null    float64 
dtypes: float64(2), object(1)
memory usage: 17.3+ KB
```

c. Mengubah Data

Sekarang saya ingin mengubah data dari sebelumnya hanya 2 fitur menjadi 4 hari sebelum yang terdapat 5 fitur (t-4, t-3, t-2, t-1, dan t sebagai label) karena dari uji korelasi, keempat fitur tersebut (t-1 sampai t-4) merupakan nilai uji korelasi terbaik (lebih dari 0.5). Saya juga membuat data 10 hari sebelum untuk membandingkan apakah semakin banyak hari sebelum, semakin baik pula modelnya?


```
supervised_df = create_supervised(df['N02_scaled'], n_lag=4)

print(supervised_df)
print(supervised_df.shape)
```

output/terminal

	N02(t-4)	N02(t-3)	N02(t-2)	N02(t-1)	N02(t)
4	0.238203	0.192840	0.196854	0.149560	0.154247
5	0.192840	0.196854	0.149560	0.154247	0.185625
6	0.196854	0.149560	0.154247	0.185625	0.152010
7	0.149560	0.154247	0.185625	0.152010	0.149143
8	0.154247	0.185625	0.152010	0.149143	0.159907
...	...	...	...	...	...
726	0.123092	0.325742	0.372653	0.145997	0.094458
727	0.325742	0.372653	0.145997	0.094458	0.089599
728	0.372653	0.145997	0.094458	0.089599	0.000000
729	0.145997	0.094458	0.089599	0.000000	0.014405
730	0.094458	0.089599	0.000000	0.014405	0.014405

[727 rows x 5 columns]
(727, 5)

Untuk membuat data 10 hari sebelum tinggal tambah code dibawah (ubah parameter n_lag).

```
supervised_df10 = create_supervised(df['N02_scaled'], n_lag=10)

print(supervised_df10)
print(supervised_df10.shape)
```

output/terminal

	N02(t-10)	N02(t-9)	N02(t-8)	N02(t-7)	N02(t-6)	N02(t-5)	N02(t-4)	N02(t-3)
10	0.238203	0.192840	0.196854	0.149560	0.154247	0.185625	0.152010	0.149143
11	0.192840	0.196854	0.149560	0.154247	0.185625	0.152010	0.149143	0.159907
12	0.196854	0.149560	0.154247	0.185625	0.152010	0.149143	0.159907	0.242292
13	0.149560	0.154247	0.185625	0.152010	0.149143	0.159907	0.242292	0.214105
14	0.154247	0.185625	0.152010	0.149143	0.159907	0.242292	0.214105	0.166781
...	...	...	...	...	...	...	...	...
726	0.161874	0.128849	0.095824	0.062799	0.038033	0.059606	0.123092	0.325742
727	0.128849	0.095824	0.062799	0.038033	0.059606	0.123092	0.325742	0.372653
728	0.095824	0.062799	0.038033	0.059606	0.123092	0.325742	0.372653	0.145997
729	0.062799	0.038033	0.059606	0.123092	0.325742	0.372653	0.145997	0.094458
730	0.038033	0.059606	0.123092	0.325742	0.372653	0.145997	0.094458	0.089599

[721 rows x 11 columns]
(721, 11)

d. Modeling dan Evaluation

Sekarang dari 2 data yang sudah kita rubah, kita train menggunakan model KNN Regression.

```

from sklearn.neighbors import KNeighborsRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error, r2_score
import numpy as np

def MAPE(y_true, y_pred):
    y_true, y_pred = np.array(y_true), np.array(y_pred)
    # Hindari pembagian dengan nol
    nonzero = y_true != 0
    return np.mean(np.abs((y_true[nonzero] - y_pred[nonzero]) / y_true[nonzero])) *

def train_knn(df_supervised, model_name=""):
    # Pisahkan fitur & label
    X = df_supervised.drop(columns=['N02(t)']).values
    y = df_supervised['N02(t)'].values

    # Split data 80/20
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, shuffle=False
    )

    # Model KNN
    knn = KNeighborsRegressor(n_neighbors=5)
    knn.fit(X_train, y_train)

    # Prediksi
    y_pred = knn.predict(X_test)

    # Evaluasi
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    r2 = r2_score(y_test, y_pred)
    mape = MAPE(y_test, y_pred)

    print(f"\n=== {model_name} ===")
    print(f"Train Size: {len(X_train)} - Test Size: {len(X_test)}")
    print(f"RMSE: {rmse:.6f}")
    print(f"R2 Score: {r2:.4f}")
    print(f"MAPE: {mape:.4f}%")

    return knn, y_test, y_pred

# Train model untuk 4 hari sebelumnya
knn_4, y_test_4, y_pred_4 = train_knn(supervised_df, "KNN - 4 Hari Sebelumnya")

# Train model untuk 10 hari sebelumnya
knn_10, y_test_10, y_pred_10 = train_knn(supervised_df10, "KNN - 10 Hari Sebelumnya")

```

```
output/terminal
=== KNN - 4 Hari Sebelumnya ===
Train Size: 581 – Test Size: 146
RMSE: 0.065436
R2 Score: 0.1395
MAPE: 61.0780%
```

```
=== KNN - 10 Hari Sebelumnya ===
Train Size: 576 – Test Size: 145
RMSE: 0.067567
R2 Score: 0.0886
MAPE: 64.6611%
```

Berdasarkan hasil akurasi diatas menunjukkan bahwa lebih banyak hari sebelumnya maka model semakin bagus. Kita coba gunakan data 30 hari sebelumnya juga untuk melihat apakah semakin banyak hari sebelumnya, model semakin baik?

```
knn_30, y_test_30, y_pred_30 = train_knn(supervised_df30, "KNN - 30 Hari Sebelumnya")

output/terminal
=== KNN - 30 Hari Sebelumnya ===
Train Size: 560 – Test Size: 141
RMSE: 0.074803
R2 Score: -0.0875
MAPE: 72.2295%
```

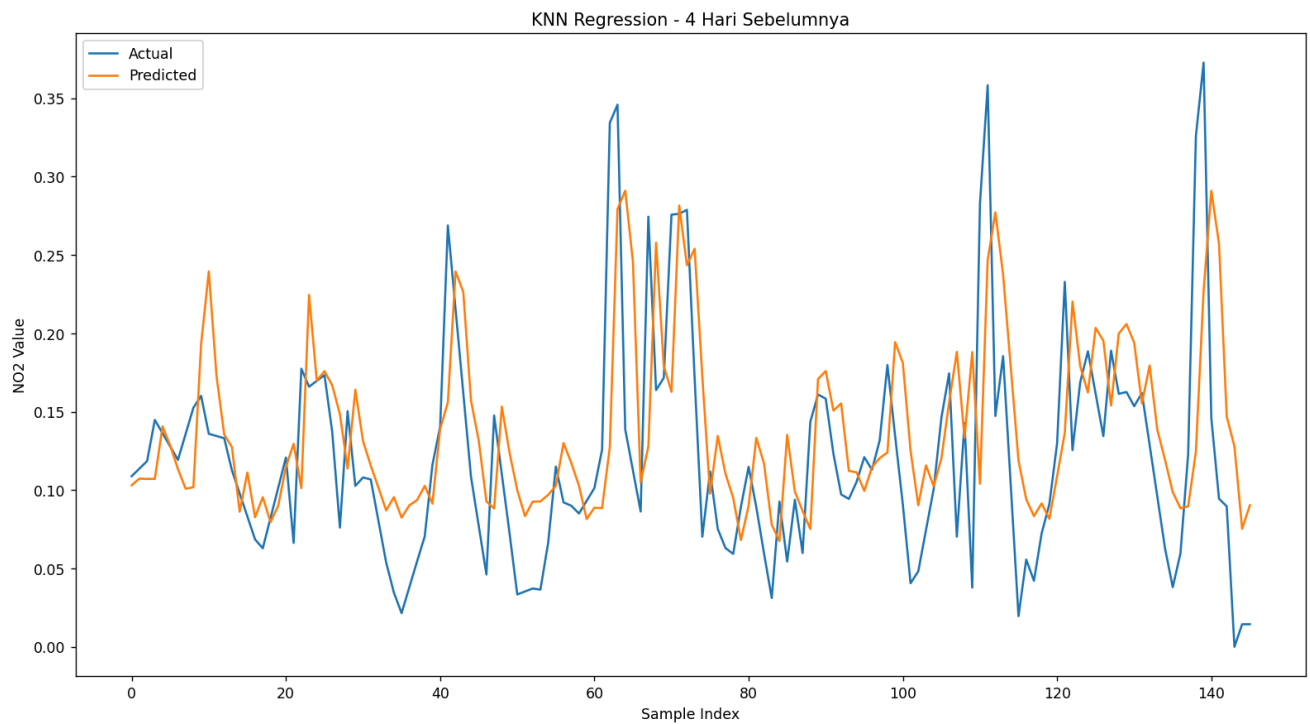
e. Plotting

Plotting untuk visualisasi grafik antara label dan prediksi dari kedua data diatas.

4 hari sebelum:

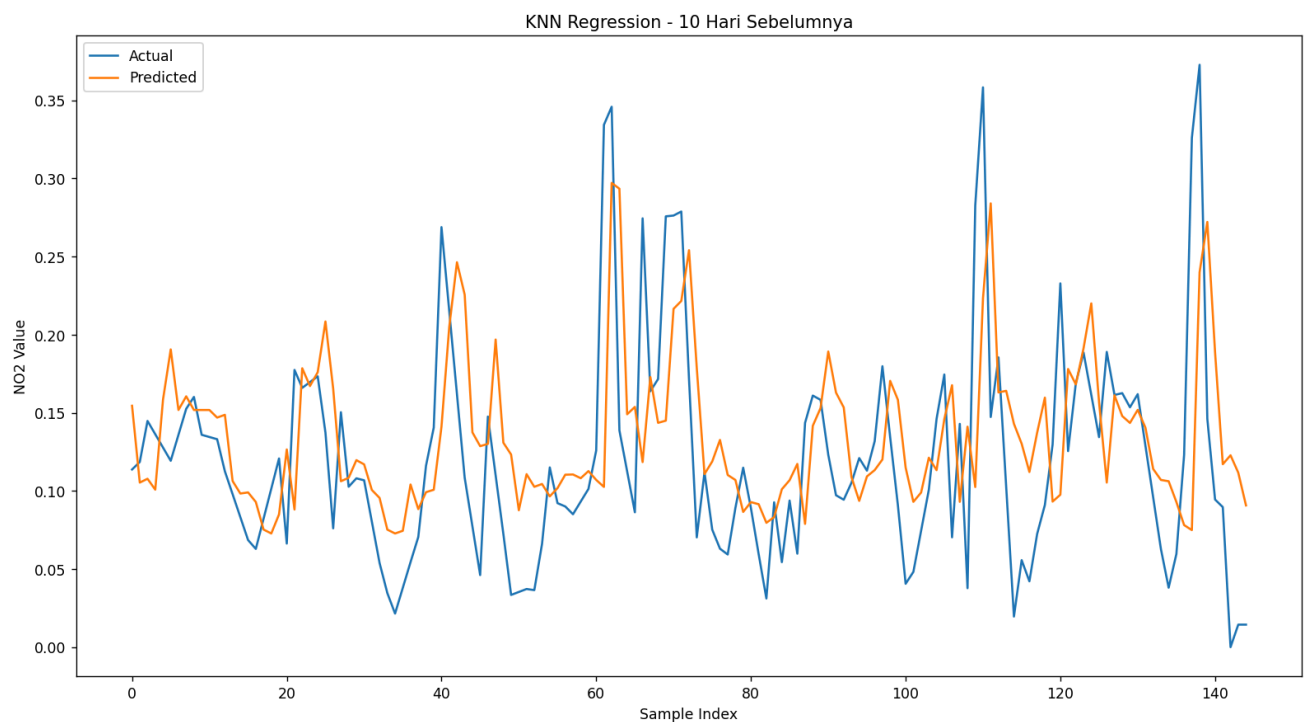
```
import matplotlib.pyplot as plt
import numpy as np

plt.figure()
plt.plot(np.arange(len(y_test_4)), y_test_4, label="Actual")
plt.plot(np.arange(len(y_pred_4)), y_pred_4, label="Predicted")
plt.title("KNN Regression - 4 Hari Sebelumnya")
plt.xlabel("Sample Index")
plt.ylabel("NO2 Value")
plt.legend()
plt.show()
```



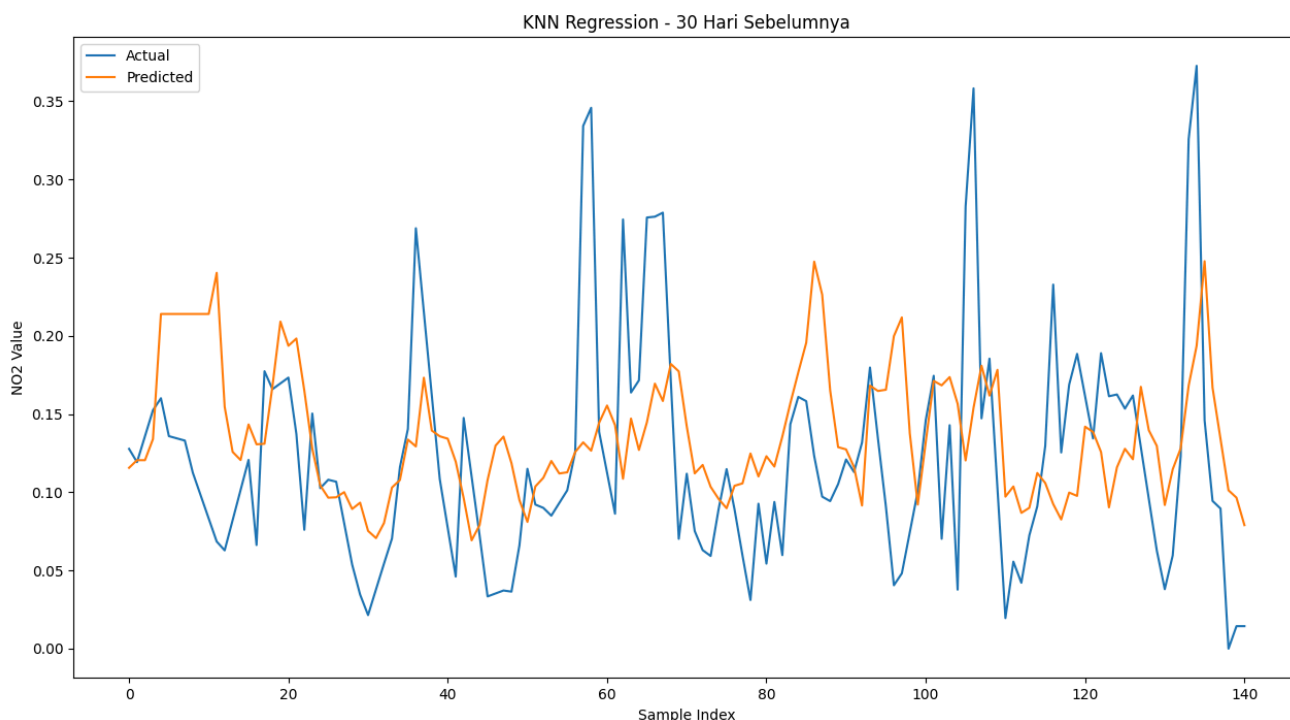
10 hari sebelum:

```
plt.figure()
plt.plot(np.arange(len(y_test_10)), y_test_10, label="Actual")
plt.plot(np.arange(len(y_pred_10)), y_pred_10, label="Predicted")
plt.title("KNN Regression - 10 Hari Sebelumnya")
plt.xlabel("Sample Index")
plt.ylabel("NO2 Value")
plt.legend()
plt.show()
```



30 hari sebelum:

```
plt.figure()
plt.plot(np.arange(len(y_test_30)), y_test_30, label="Actual")
plt.plot(np.arange(len(y_pred_30)), y_pred_30, label="Predicted")
plt.title("KNN Regression - 30 Hari Sebelumnya")
plt.xlabel("Sample Index")
plt.ylabel("NO2 Value")
plt.legend()
plt.show()
```



Hasil evaluasi model KNN Regression menunjukkan bahwa peningkatan jumlah fitur historis (lag) tidak serta merta meningkatkan performa prediksi. Pada model dengan 4 hari sebelumnya, nilai RMSE paling kecil dan R^2 masih positif sehingga model mampu menjelaskan sebagian kecil variabilitas data target. Namun, ketika jumlah lag ditambah menjadi 10 dan 30 hari sebelumnya, performa model justru menurun yang ditunjukkan oleh meningkatnya nilai RMSE dan MAPE, serta penurunan nilai R^2 hingga bernilai negatif pada lag 30. Nilai MAPE yang cukup tinggi pada seluruh model (lebih dari 60%) juga mengindikasikan bahwa akurasi prediksi masih rendah dan terdapat deviasi besar antara nilai prediksi dan nilai aktual. Secara keseluruhan, model KNN tidak memberikan performa yang baik pada data ini, dan penambahan fitur historis justru menyebabkan overfitting serta menurunkan kemampuan generalisasi model. Oleh karena itu, diperlukan pemilihan model lain atau peningkatan strategi preprocessing untuk memperoleh hasil prediksi yang lebih baik.